Section 05: DB & Validation

堅牢なバックエンドとデータベース操作

データベースへのアクセスと、それを型安全に扱う技術を学びます。

1. ユーザーの入力は「疑う」ことから始まる

前回の Section 04 では、`c.req.json<T>()` を使ってリクエストに型を付けました。しかし、これだけでは不十分です。

- **TypeScript は「実行時」には無力**
  - **型消去 (Type Erasure)**: TypeScript は開発中にエラーを見つけるためのもので、JavaScript に変換されると型情報はすべて消えます。
  - **`<T>` は値をチェックしない**: `c.req.json<User>()` と書いても、送られてきたデータの中身を TypeScript が自動で検証することはありません。
  - **実行時エラーの危険**: 文字列を期待している場所に数値が届くなど、想定外のデータが入り込むとアプリがクラッシュする原因になります。

「開発時の安心」と「実行時の安全」

	タイミング	役割	守ってくれるもの
TypeScript	コンパイル時	開発者のタイポやミスを防ぐ	コードを書いている自分
バリデーター	実行時	外部からの不正なデータを弾く	稼働しているシステム

バリデーションが必要な「不正データ」の例

- `age` がマイナス（-100）や、ありえない数値（10000）で送られてきたら？
- `email` の形式がデタラメ（`abc@@example`）だったら？
- 必須のはずの項目が空文字（`""`）だったら？

これらを防ぐのが **バリデーション（検証）** です。

Zod：TypeScript 時代の「標準バリデーター」

現代の TypeScript 開発において、データの検証（バリデーション）でデファクトスタンダードとなっているのが **Zod** です。

- **型と検証の一致**: スキーマ（定義）を書くと、それに対応する TypeScript の「型」を自動生成できます。
- **直感的な記述**: 「文字列で、1文字以上、メール形式」といったルールを `.` （ドット）でつなげて直感的に書けます。
- **エコシステムの中心**: Hono, Prisma, React Hook Form など、あらゆるツールが Zod をサポートしています。

Zod の基本文法

「どんなデータが欲しいか」をオブジェクト形式で定義します。

```
import { z } from "zod";

// 「データの形」の定義（スキーマ）
const UserSchema = z.object({
  name: z.string().min(1, "名前は必須です"), // 1文字以上
  age: z.number().int().min(0).max(120), // 0〜120の整数
  email: z.string().email("形式が不正です"), // メール形式
  tags: z.array(z.string()).optional(), // 文字列の配列、省略OK
});

// スキーマから自動的に TypeScript の型を取り出すことも可能！
type User = z.infer<typeof UserSchema>;
```

`z.infer`：定義と型を一本化する

```
type User = z.infer<typeof UserSchema>;
```

Zod の最も強力な機能の一つが、この「型推論」です。

- **重複を排除**: `interface` と `Schema` を別々に書く必要がありません。
- **一貫性の保証**: スキーマ（ルール）を修正すれば、自動的に TypeScript の型も更新されます。
- **Single Source of Truth**: 「バリデーションルール」がそのまま「型の定義」になるため、定義のズレによるバグを根絶できます。

【クイズ】 Zod スキーマを読み解こう

次のスキーマとデータを見て、バリデーションが**成功するか失敗するか**考えてみよう。

スキーマ:

```
const QuizSchema = z.object({
  id: z.number().int().positive(),
  username: z.string().min(3).max(10),
  role: z.enum(["admin", "user"]),
});
```

1. `{ id: 10, username: "dev", role: "guest" }` → 成功？ 失敗？
2. `{ id: -1, username: "alice", role: "admin" }` → 成功？ 失敗？
3. `{ id: 5, username: "hi", role: "user" }` → 成功？ 失敗？

答え

```
const QuizSchema = z.object({
  id: z.number().int().positive(),
  username: z.string().min(3).max(10),
  role: z.enum(["admin", "user"]),
});
```

1. `{ id: 10, username: "dev", role: "guest" }` → **失敗**（`role` が `admin` か `user` である必要がある）
2. `{ id: -1, username: "alice", role: "admin" }` → **失敗**（`id` が `positive`（正の数）である必要がある）
3. `{ id: 5, username: "hi", role: "user" }` → **失敗**（`username` が `min(3)`（3文字以上）である必要がある）

【逆引きクイズ】 ルールからスキーマを作ろう

「20文字以内の文字列で、中身が空（""）なのはNG」という名前（`name`）のルールを Zod で書くとどうなる？

- `z.string()` で文字列であることを宣言します
- `min(n)` で最小文字数、`max(n)` で最大文字数を指定できます
- 先ほどの「`.`（ドット）でつなげる」書き方を思い出してみましょう

こたえ

```
z.string().min(1).max(20);
```

または

```
z.string().nonempty().max(20);
```

Hono Middleware：関所の概念

Zod Middleware Flow

実際の「処理（ハンドラ）」にたどり着く前に、データをチェックする **「関所（ミドルウェア）」** という仕組みがあります。

1. **Request**: クライアントからデータが届く
2. **Middleware**: **ここで Zod を使ってデータを検閲する**
   - 不正なデータ：ここで追い返し、ハンドラには通さない（400 Bad Request）
   - 正しいデータ：次のハンドラへ通す
3. **Handler**: 検証済みの安全なデータを使って、DB保存などのメイン処理を行う

ミドルウェアの最もシンプルな形

「バリデーション」もミドルウェアの一種ですが、もっと身近な例で動きを見てみましょう。例えば、**「合言葉を知っている人だけ通す関所」** を作るとしたらこうなります。

```
app.get(
  "/secret", // ←パス
  async (c, next) => {
    // ←ミドルウェアの開始
    // [1] 前処理：合言葉をチェックする
    if (c.req.query("pass") !== "shakemake") {
      return c.text("合言葉が違います！", 401); // 追い返す
    }
    // [2] 合格！次の人（ハンドラ）へバトンタッチ
    await next();
  }, // ←ミドルウェアの終了
  (c) => {
    // [3] メインの処理：ここには「合格した人」しか来ない
    return c.text("秘密のメッセージ：IT部へようこそ！");
  },
);
```

- `next()`：「チェック完了！次へ進んでよし！」という合図です。
- これを「Zod でのチェック」に置き換えたものが、この後紹介するバリデーション用ミドルウェアです。

`sValidator`：最新のバリデーション・ミドルウェア

Hono で Zod を使うには、`@hono/standard-validator` の `sValidator` を使うことができます。

```
import { sValidator } from "@hono/standard-validator";

app.post(
  "/register",
  // ハンドラの前に「関所」として sValidator を置く
  sValidator("json", UserSchema),
  async (c) => {
    // ここに来たときは、すでに検証済みのクリーンなデータ！
    const data = c.req.valid("json");
    return c.json({ message: "OK", user: data });
  },
);
```

> **なぜ "Standard"？**: Zod 以外のライブラリ（Valibot 等）も同じ仕様で扱える「標準（Standard Schema）」に準拠しているためです。

> **便利なデフォルト動作**: `sValidator` は、バリデーションに失敗した場合、自動的に **400 Bad Request** のステータスコードと共に、どこがどう間違っていたか（例：「名前は必須です」）を詳細な JSON エラーとして返してくれます。エラー時のハンドリングを自分で書く必要はありません。

2. Drizzle ORM × Zod：SQL にも「型」の力を

D1（SQL）をそのまま使うと？

Cloudflare D1 は SQL で操作しますが、そのまま開発するといくつか課題があります。

- **スペルミスに気付かない**: `SELECT * FROM usres`（正しくは `users`）と書いても、実行するまでエラーになりません。
- **データ構造がわからない**: 「このテーブル、どんなカラムがあったっけ？」といちいち SQL を確認しに行く必要があります。
- **型変換の二度手間**: DB から取得したデータに、毎回 TypeScript の型を自分で付けるのは大変です。

> このような問題を解決するために、 **Drizzle ORM** を使います。「TypeScript で SQL を書く」感覚で、100% 型安全な開発が可能になります。

Drizzle でテーブルを定義する

データベースの形（スキーマ）を定義します。

```
import { sqliteTable, text, integer } from "drizzle-orm/sqlite-core";

// ユーザーテーブルの定義
export const users = sqliteTable("users", {
  id: integer("id").primaryKey({ autoIncrement: true }),
  name: text("name").notNull(),
  email: text("email").notNull().unique(),
  age: integer("age"),
});
```

- **TypeScript が SQL を理解**: `users.` と打てば、カラム名（`name`, `email` など）が補完されます。
- **型情報の抽出**: この定義から、Zod スキーマや TypeScript の型を自動生成できます。

drizzle-zod：究極の自動化

Drizzle と Zod を組み合わせると、バリデーションルールの作成がさらに楽になります。

```
import { createInsertSchema } from "drizzle-zod";
import { users } from "./db/schema";

// Drizzle の定義から、Zod スキーマを自動生成！
const UserInsertSchema = createInsertSchema(users);

// もちろん、特定の項目にルールを追加することも可能
const DetailedUserSchema = createInsertSchema(users, {
  email: z.string().email(),
  age: z.number().min(0).max(120),
});
```

データの操作 (SELECT / INSERT)

Drizzle を使うと、SQL が直感的な TypeScript のメソッド呼び出しになります。

```
import { drizzle } from "drizzle-orm/d1";

app.get("/users", async (c) => {
  // 環境変数から DB 接続を取得
  const db = drizzle(c.env.DB);

  // SELECT * FROM users
  const allUsers = await db.select().from(users).all();

  return c.json(allUsers);
});
```

なぜ D1 × Drizzle なのか？

1. **開発速度**: カラム名の補完が効くので、タイポで悩む時間がゼロになります。
2. **安全な変更**: テーブルの構造を変えたら、それを参照しているコードがすべてコンパイルエラーになるため、壊れたままリリースするのを防げます。
3. **Zod との相性**: DB の定義がバリデーションルールに直結するため、不整合が起きません。

2-1. ワークショップ：掲示板 API 構築 (D1+Drizzle版)

少し難易度が高いと感じるかもしれませんが、以下の通りに実装すれば大丈夫です！「設計図」から「完成品」までをセットで見ていきましょう。

0. データベースの初期化 (SQL)

Drizzle を使う前に、D1 データベースに「テーブル」を作る必要があります。
`/schema.sql` というファイルを作り、以下の SQL を保存してコマンドを実行しましょう。

```sql
-- schema.sql
CREATE TABLE messages (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  content TEXT NOT NULL
);
```

実行コマンド:

```
npx wrangler d1 execute <データベース名> --local --file=./schema.sql
```

📋 1. Drizzle での設計図 (Schema)

まずは「どんな情報を保存するか」を決めます。

- テーブル名: `messages`
- 項目: `id` (自動採番), `name` (名前), `content` (本文)

```
import { sqliteTable, text, integer } from "drizzle-orm/sqlite-core";

const messages = sqliteTable("messages", {
  id: integer("id").primaryKey({ autoIncrement: true }),
  name: text("name").notNull(),
  content: text("content").notNull(),
});
```

2. 実装する機能 (Goals)

この 2 つの機能を作れば完成です！

1. **メッセージを保存する (`POST /messages`)**
    - 名前と本文を JSON で受け取る
    - Zod で「空っぽじゃないか」をチェックする
    - D1 データベースに保存する
2. **メッセージを全部見る (`GET /messages`)**
    - D1 からこれまでの投稿を全部取ってくる
    - JSON で画面に返す

3. KV (Key-Value) ストレージの活用

KV とは？

SQLデータベース以外にも、**KV (Key-Value)** というデータストアがあります。

- **SQL (RDB)**: 複雑な関係性、検索、集計が必要なデータ（投稿一覧など）。
- **KV (Key-Value)**: 単純な「合言葉（キー）」と「中身（値）」のペア。
  - **メリット**: SQL より圧倒的に読み取りが高速。
  - **用途**: 設定値、一時的なキャッシュ、ランキングデータ。

例

キー (Key)	値 (Value)	説明
`is_maintenance`	`"true"`	メンテナンス中かどうかのフラグ
`config_theme`	`"dark"`	アプリ全体の着せ替え設定
`user:101:session`	`"abc-123-xyz"`	ログイン中ユーザーのセッション情報

KV ストレージへのアクセス

KV ストアはシンプルな `get` / `put` / `delete` で操作できます。

```
app.get("/status", async (c) => {
  // KV からデータを取得
  const kvStore = c.env.KV;
  const maintenanceStatus = await kvStore.get("maintenance_mode");

  return c.json({ maintenance: maintenanceStatus === "true" });
});

app.post("/status", async (c) => {
  // KV にデータを保存
  const kvStore = c.env.KV;
  await kvStore.put("maintenance_mode", "true");
  return c.text("Updated!");
});
```